ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC ỨNG DỤNG

BÁO CÁO BÀI TẬP LỚN MÔN VẬT LÍ

BÀI 10: XÁC ĐỊNH LỰC TƯƠNG TÁC TĨNH ĐIỆN GIỮA CÁC ĐIỆN TÍCH ĐIỂM TRONG CHÂN KHÔNG

GVHD: Nguyễn Văn Khá

Tên Lớp/Tên Nhóm: Lớp L02/Nhóm 10

Sinh viên thực hiện

Họ tên	**MSSV**
Nguyễn Thảo Vi	2610353
Nguyễn Hoàng Minh	2613005
Nguyễn Minh Nguyên	2611431

TP. HCM, 2026

Mục lục

1 MỞ ĐẦU

1.1 Cơ sở hình thành đề tài

Hiểu rõ các hiện tượng tự nhiên và kỹ thuật: Lực điện và điện trường đóng vai trò quan trọng trong nhiều hiện tượng tự nhiên (sét, hiện tượng nhiễm điện do cọ xát) cũng như các ứng dụng công nghệ hiện đại (máy lọc bụi tĩnh điện, công nghệ sơn tĩnh điện, máy in laser, các linh kiện vi điện tử).

Giải quyết bài toán thực tế nhiều vật: Trong kỹ thuật và khoa học vật liệu, việc tính toán lực tương tác giữa nhiều hạt mang điện là bước cơ bản để mô phỏng cấu trúc nguyên tử, phân tử hoặc các hệ thống vi cơ điện tử (MEMS).

Tuy nhiên, khi số lượng điện tích tăng lên (n ≥ 3), việc tính toán bằng tay trở nên phức tạp, tốn thời gian và dễ dẫn đến sai sót do các phép biến đổi đại số vectơ. Do đó, việc ứng dụng các công cụ tính toán ký hiệu (Symbolic Math) trên các ngôn ngữ lập trình như MATLAB hoặc Python giúp mô phỏng, tự động hóa và trực quan hóa các thành phần lực Fx, Fy một cách nhanh chóng và chính xác.

1.2 Mục tiêu của đề tài

- Xây dựng thuật toán và chương trình (bằng MATLAB hoặc Python) ứng dụng phép toán hình thức (symbolic) để tính toán lực tương tác tĩnh điện.
- Xử lý bài toán hệ n điện tích điểm trong mặt phẳng Oxy: Nhập tọa độ (x_i, y_i) và điện tích qiá trị, tự động xuất biểu thức và giá trị các thành phần lực F_x, F_y tác dụng lên từng điện tích.
- Trực quan hóa hệ điện tích và vectơ lực tổng hợp tác dụng lên từng điện tích bằng đồ họa.

1.3 Ý nghĩa thực tiễn của đề tài

- Cung cấp công cụ học tập và kiểm tra kết quả nhanh chóng cho sinh viên khi làm bài tập Điện học.
- Rèn luyện kỹ năng lập trình mô phỏng vật lý, tư duy thuật toán và sử dụng thư viện đại số ký hiệu (SymPy / Symbolic Toolbox).

1.4 Phạm vi giới hạn của đề tài

- Hệ điện tích được xét là các điện tích điểm đặt cố định trong môi trường chân không (môi trường đẳng hướng, hằng số điện môi ε = 1).
- Không không gian khảo sát hạn chế trong mặt phẳng tọa độ hai chiều Oxy.

2 CƠ SỞ LÝ THUYẾT

2.1 Nêu những khái niệm, vấn đề lý thuyết có liên quan

2.1.1 Định luật Coulomb

Lực tương tác tĩnh điện giữa hai điện tích điểm q_1 và q_2 đặt trong chân không cách nhau khoảng r_{12} có độ lớn tỉ lệ thuận với tích độ lớn các điện tích và tỉ lệ nghịch với bình phương khoảng cách giữa chúng:

$$\vec{F}_{12} = k\frac{|q_1 q_2|}{r_{12}^2}\vec{e}_r$$

Trong đó $k = \frac{1}{4\pi\varepsilon_0} \approx 9 \times 10^9\ \mathrm{N \cdot m^2/C^2}$.

2.1.2 Nguyên lí chồng chất lực tĩnh điện

Lực tổng hợp tĩnh điện do n điện tích điểm tác dụng lên một điện tích q_0 bằng tổng các vectơ lực do từng điện tích q_i tác dụng riêng rẽ lên q_0.

Biểu thức lực tổng hợp $\vec{F}_i$ tác dụng lên điện tích thứ i và các thành phần theo hệ tọa độ Oxy. Trong đó khoảng cách giữa điện tích q_i và q_j được tính là:

$$r_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

2.2 Mô hình hóa hoặc hệ thống hóa vấn đề nghiên cứu

Mô hình hóa hệ n điện tích điểm trong mặt phẳng Oxy:
Xét điện tích q_i tại vị trí (x_i, y_i) và q_j tại vị trí (x_j, y_j). Vectơ khoảng cách $\vec{r}_{ij} = (x_j - x_i)\vec{i} + (y_j - y_i)\vec{j}$.
Khoảng cách:

$$r_{ij} = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$$

Vectơ lực do q_i tác dụng lên q_j chiếu lên trục Ox và Oy có dạng biểu thức đại số:

$$F_{ij,x} = k \cdot \frac{q_i q_j (x_j - x_i)}{[(x_j - x_i)^2 + (y_j - y_i)^2]^{3/2}}$$
$$F_{ij,y} = k \cdot \frac{q_i q_j (y_j - y_i)}{[(x_j - x_i)^2 + (y_j - y_i)^2]^{3/2}}$$

2.3 Xây dựng giả thiết

- **Giả thiết 1:** Tương tác tĩnh điện tuân theo định luật III Newton, nghĩa là tổng lực tương tác tĩnh điện của toàn hệ kín phải bằng 0:
$$\sum \vec{F}_j = \vec{0}$$
- **Giả thiết 2:** Phép toán hình thức (Symbolic) cho phép giữ nguyên dạng biểu thức chính xác tuyệt đối trước khi thế số, không gây ra lỗi tích lũy do làm tròn số lẻ.

3 PHƯƠNG PHÁP NGHIÊN CỨU HOẶC THIẾT KẾ NGHIÊN CỨU

3.1 Chọn phương pháp

Lựa chọn phương pháp mô phỏng số kết hợp tính toán ký hiệu (Symbolic Computing) bằng ngôn ngữ Python (sử dụng thư viện SymPy, NumPy, Matplotlib) hoặc MATLAB.

3.2 Nhu cầu thông tin và Nguồn thông tin

- **Nhu cầu thông tin:** Công thức định luật Coulomb, thuật toán tính tổng đại số vectơ, tài liệu hướng dẫn thư viện SymPy/MATLAB.
- **Nguồn thông tin:** Giáo trình Vật lý đại cương A1 (Bộ môn Vật lý Ứng dụng – ĐH Bách Khoa TP.HCM), tài liệu *MATLAB Projects for Scientists and Engineers* (A. L. Garcia).

3.3 Thiết kế mẫu (Thiết kế bài toán & dữ liệu)

Thiết kế mẫu dữ liệu kiểm thử gồm hệ 3 điện tích điểm đặt tại các đỉnh của tam giác vuông trong mặt phẳng Oxy:

- $q_1 = +2\ \mu\text{C}$ tại vị trí $(0;\ 0)$ m
- $q_2 = -3\ \mu\text{C}$ tại vị trí $(0{,}3;\ 0)$ m
- $q_3 = +4\ \mu\text{C}$ tại vị trí $(0;\ 0{,}4)$ m

3.4 Phương pháp và công cụ thu thập thông tin

Chương trình Python tự động nhận dữ liệu đầu vào qua bàn phím gồm số điện tích n, giá trị q_i và tọa độ (x_i, y_i). Chương trình tính các thành phần lực Coulomb do từng điện tích tác dụng, cộng vectơ lực để tìm lực tổng hợp, in kết quả và vẽ sơ đồ hệ điện tích.

Mã nguồn được trình bày dưới đây:

```python
import math
from pathlib import Path
import sympy as sp
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.transforms import blended_transform_factory
def tinh_luc_he_dien_tich(danh_sach_dien_tich):
    n = len(danh_sach_dien_tich)
    k_sym = sp.Symbol('k')
    k_val = 9e9
    ket_qua = {}

    for i in range(n):
        q_i = danh_sach_dien_tich[i]['q']
        x_i = danh_sach_dien_tich[i]['x']
        y_i = danh_sach_dien_tich[i]['y']

        Fx_expr, Fy_expr = 0, 0
        Fx_val, Fy_val = 0.0, 0.0
        Fx_components = []
        Fy_components = []

        for j in range(n):
            if i == j:
                continue
            q_j = danh_sach_dien_tich[j]['q']
            x_j = danh_sach_dien_tich[j]['x']
            y_j = danh_sach_dien_tich[j]['y']

```

```
            dx = x_i - x_j
            dy = y_i - y_j
            r2 = dx**2 + dy**2

            if r2 > 0:
                r = math.sqrt(r2)
                r3 = r2 * r

                # Gi  itr ố
                f_x = k_val * (q_i * q_j * dx) / r3
                f_y = k_val * (q_i * q_j * dy) / r3
                Fx_val += f_x
                Fy_val += f_y

                # ểBiu ứthc SymPy
                fx_component = k_sym * (q_i * q_j * dx) / (r2 ** sp.Rational(3, 2))
                fy_component = k_sym * (q_i * q_j * dy) / (r2 ** sp.Rational(3, 2))
                Fx_expr += fx_component
                Fy_expr += fy_component
                component_id = len(Fx_components) + 1
                Fx_components.append({
                    'name': f'Fx{component_id}',
                    'source_id': j + 1,
                    'expr': sp.simplify(fx_component),
                    'value': f_x,
                    'vector_x': f_x,
                    'vector_y': f_y
                })
                Fy_components.append({
                    'name': f'Fy{component_id}',
                    'source_id': j + 1,
                    'expr': sp.simplify(fy_component),
                    'value': f_y
                })

        F_total = math.hypot(Fx_val, Fy_val)
        goc_do = math.degrees(math.atan2(Fy_val, Fx_val)) % 360

        ket_qua[i + 1] = {
            'Fx_expr': sp.simplify(Fx_expr),
            'Fy_expr': sp.simplify(Fy_expr),
            'Fx_components': Fx_components,
            'Fy_components': Fy_components,
            'Fx_val': Fx_val,
            'Fy_val': Fy_val,
            'F_total': F_total,
            'goc_do': goc_do
        }

    return ket_qua

# ========================================
# H M ẼV ĐỒ ỊTH ƠVECT ỰLC
# ========================================
def ve_so_do_dien_tich(danh_sach, ket_qua, ten_tep='so_do_dien_tich.png'):
    fig, ax = plt.subplots(figsize=(20, 12))

    x_values = [dt['x'] for dt in danh_sach]
    y_values = [dt['y'] for dt in danh_sach]
    plot_span = max(
        max(x_values + [0]) - min(x_values + [0]),
        max(y_values + [0]) - min(y_values + [0])
    )
    if plot_span == 0:
        plot_span = 1.0
    max_vector_F = max(
        [
            item['F_total']
```

```
            for item in ket_qua.values()
        ] + [
            math.hypot(component['vector_x'], component['vector_y'])
            for item in ket_qua.values()
            for component in item['Fx_components']
        ],
        default=0.0
    )
    nearest_distances = []
    for i, dt in enumerate(danh_sach):
        distances = [
            math.hypot(dt['x'] - other['x'], dt['y'] - other['y'])
            for j, other in enumerate(danh_sach)
            if i != j and math.hypot(dt['x'] - other['x'], dt['y'] - other['y']) > 0
        ]
        if distances:
            nearest_distances.append(min(distances))
    typical_spacing = (
        sorted(nearest_distances)[len(nearest_distances) // 2]
        if nearest_distances else plot_span
    )
    arrow_length = min(plot_span * 0.4, typical_spacing * 0.6)
    force_scale = arrow_length / max_vector_F if max_vector_F > 0 else 0.0
    arrow_endpoints = []

    for i, dt in enumerate(danh_sach):
        for dt_ke in danh_sach[i + 1:]:
            plt.plot(
                [dt['x'], dt_ke['x']],
                [dt['y'], dt_ke['y']],
                color='gray',
                linestyle='--',
                linewidth=0.8,
                alpha=0.55,
                zorder=1
            )

    for i, dt in enumerate(danh_sach):
        q, x, y = dt['q'], dt['x'], dt['y']
        dt_id = i + 1
        luc = ket_qua[dt_id]

        scaled_components = [
            (
                component,
                component['vector_x'] * force_scale,
                component['vector_y'] * force_scale
            )
            for component in luc['Fx_components']
        ]
        resultant_dx = sum(dx for _, dx, _ in scaled_components)
        resultant_dy = sum(dy for _, _, dy in scaled_components)
        resultant_endpoint = (
            x + resultant_dx,
            y + resultant_dy
        )

        if len(scaled_components) == 2:
            component_endpoints = []
            for component, dx_component, dy_component in scaled_components:
                endpoint = (x + dx_component, y + dy_component)
                component_endpoints.append(endpoint)
                arrow_endpoints.append(endpoint)
                plt.quiver(
                    x, y, dx_component, dy_component,
                    angles='xy', scale_units='xy', scale=1,
                    color='#c2410c', alpha=0.9, width=0.0022,
                    headwidth=4, headlength=5, headaxislength=4, zorder=2
```

```
                )
            for endpoint in component_endpoints:
                plt.plot(
                    [endpoint[0], resultant_endpoint[0]],
                    [endpoint[1], resultant_endpoint[1]],
                    color='#c2410c',
                    linestyle=':',
                    linewidth=0.9,
                    alpha=0.7,
                    zorder=1
                )
        else:
            polygon_x, polygon_y = x, y
            for component, dx_component, dy_component in scaled_components:
                next_x = polygon_x + dx_component
                next_y = polygon_y + dy_component
                arrow_endpoints.append((next_x, next_y))
                plt.quiver(
                    polygon_x, polygon_y, dx_component, dy_component,
                    angles='xy', scale_units='xy', scale=1,
                    color='#c2410c', alpha=0.9, width=0.0022,
                    headwidth=4, headlength=5, headaxislength=4, zorder=2
                )
                polygon_x, polygon_y = next_x, next_y
            if len(scaled_components) > 2:
                plt.plot(
                    [polygon_x, x],
                    [polygon_y, y],
                    color='#c2410c',
                    linestyle=':',
                    linewidth=0.9,
                    alpha=0.7,
                    zorder=1
                )

        # 1. ẽV ịv tr  đệin t ch (ươDng: Đỏ,  m: Xanh)
        color = 'red' if q > 0 else 'blue'
        plt.scatter(x, y, color=color, s=120, zorder=3, edgecolors='black', linewidths=0.8)
        plt.annotate(
            f"({x:g}; {y:g})",
            xy=(x, y),
            xytext=(0, 13),
            textcoords='offset points',
            ha='center',
            va='bottom',
            fontsize=10,
            color='black'
        )
        plt.annotate(
            rf"$q_{{{dt_id}}}$",
            xy=(x, y),
            xytext=(10, -5),
            textcoords='offset points',
            ha='left',
            va='top',
            fontsize=10,
            color=color
        )

        # 2. ẽV ơVect ổTng ựlc F t c ụdng l n đệin t ch
        if math.hypot(resultant_dx, resultant_dy) > 0 and force_scale > 0:
            fy_operator = '+' if luc['Fy_val'] >= 0 else '-'
            dx_vec = resultant_dx
            dy_vec = resultant_dy
            arrow_endpoints.append((x + dx_vec, y + dy_vec))

            plt.quiver(
                x, y, dx_vec, dy_vec,
```

```
                angles='xy', scale_units='xy', scale=1,
                color='darkgreen', width=0.0028,
                headwidth=4, headlength=5, headaxislength=4, zorder=5
            )

            # Ghi ch  th ch ổtng ựlc F ạcnh ũmi t n
            plt.annotate(
                rf"$\vec{{F}}_{{{dt_id}}}$",
                xy=(x + dx_vec, y + dy_vec),
                xytext=(12 if dx_vec >= 0 else -12, 12 if dy_vec >= 0 else -12),
                textcoords='offset points',
                ha='left' if dx_vec >= 0 else 'right',
                va='bottom' if dy_vec >= 0 else 'top',
                color='darkgreen',
                fontsize=9,
                weight='normal',
                bbox={'facecolor': 'white', 'edgecolor': 'none', 'alpha': 0.8, 'pad': 1.5},
                annotation_clip=False
            )

    plt.grid(True, linestyle=':', alpha=0.6)
    plt.xlabel('ọTa độ X (m)')
    plt.ylabel('ọTa độ Y (m)')
    ax.set_title(
        'ỚS ĐỒ ỆH ĐỆIN T CH V  ỚVECT ỔTNG ỰLC\n'
        '(Cam: ựlc th nh ầphn; xanh l : ựlc ổtng ợhp; n t đứt: ốni đệin t ch)',
        fontsize=12,
        weight='bold'
    )
    ax.legend(
        handles=[
            Line2D([0], [0], color='#c2410c', linewidth=1.2, label='F ồ→đngun ch (v  ựd →F
: q t c ựdng l n q)'),
            Line2D([0], [0], color='darkgreen', linewidth=1.2, label='ựLc ổtng ợhp')
        ],
        loc='upper right',
        fontsize=9
    )
    all_x = [0] + x_values + [point[0] for point in arrow_endpoints]
    all_y = [0] + y_values + [point[1] for point in arrow_endpoints]
    padding = plot_span * 0.3
    x_min, x_max = min(all_x) - padding, max(all_x) + padding
    y_min, y_max = min(all_y) - padding, max(all_y) + padding
    y_span = y_max - y_min
    ax.set_position([0.05, 0.1, 0.68, 0.82])
    min_x_span = y_span * 1.4
    x_center = (x_min + x_max) / 2
    if x_max - x_min < min_x_span:
        x_min = x_center - min_x_span / 2
        x_max = x_center + min_x_span / 2
    ax.set_xlim(x_min, x_max)
    ax.set_ylim(y_min, y_max)
    ax.set_aspect('equal', adjustable='box')
    ax.axhline(0, color='black', linewidth=1.1, zorder=3)
    ax.axvline(0, color='black', linewidth=1.1, zorder=3)
    x_axis_transform = blended_transform_factory(ax.transAxes, ax.transData)
    y_axis_transform = blended_transform_factory(ax.transData, ax.transAxes)
    ax.annotate(
        '',
        xy=(1, 0),
        xytext=(0, 0),
        xycoords=x_axis_transform,
        textcoords=x_axis_transform,
        arrowprops={'arrowstyle': '->', 'color': 'black', 'linewidth': 1.2},
    )
    ax.annotate(
        '',
        xy=(0, 1),
```

```
        xytext=(0, 0),
        xycoords=y_axis_transform,
        textcoords=y_axis_transform,
        arrowprops={'arrowstyle': '->', 'color': 'black', 'linewidth': 1.2},
    )
    ax.annotate('x', xy=(1, 0), xycoords=x_axis_transform, xytext=(-5, 8),
                textcoords='offset points', ha='right', va='bottom', fontsize=12, weight='bold')
    ax.annotate('y', xy=(0, 1), xycoords=y_axis_transform, xytext=(8, -5),
                textcoords='offset points', ha='left', va='top', fontsize=12, weight='bold')
    fig.text(0.75, 0.88, 'ỰLC ỔTNG ỤHP', fontsize=14, weight='bold', color='darkgreen')
    row_height = min(0.19, 0.7 / max(len(ket_qua), 1))
    for row, (dt_id, luc) in enumerate(ket_qua.items()):
        fy_operator = '+' if luc['Fy_val'] >= 0 else '-'
        component_sources = ', '.join(
            rf"$F_{{{c['source_id']}\to{dt_id}}}$"
            for c in luc['Fx_components']
        ) or 'Kh ng c  th nh ảphn ựlc'
        y_text = 0.82 - row * row_height
        fig.text(
            0.75, y_text,
            f"q{dt_id} ịchu c c ựlc m u cam:\n{component_sources}",
            fontsize=11,
            color='#c2410c',
            va='top'
        )
        fig.text(
            0.75, y_text - 0.045,
            rf"$\vec{{F}}_{{{dt_id}}}=({luc['Fx_val']:.2e})\vec{{i}}"
            rf"{fy_operator}({abs(luc['Fy_val']):.2e})\vec{{j}}$",
            fontsize=11,
            color='darkgreen',
            va='top',
            bbox={'facecolor': 'white', 'edgecolor': 'none', 'alpha': 0.8, 'pad': 1}
        )
    output_path = Path(__file__).resolve().with_name(ten_tep)
    plt.savefig(output_path, dpi=150, bbox_inches='tight')
    print(f"Đ  ưlu h nh ạti: {output_path}")
    plt.show()

# ========================================
# ƯƠCHNG TR NH CH NH
# ========================================
def nhap_du_lieu_mau():
    # ữD ệliu ẫmu (3 điện t ch ếxp th nh tam gi c) để test nhanh
    return [
        {'q': 2e-6,  'x': 0.0, 'y': 0.0},   # q1 ươdng ở ốgc ọta độ
        {'q': -4e-6, 'x': 3.0, 'y': 0.0},   # q2  m ở (3, 0)
        {'q': 3e-6,  'x': 1.5, 'y': 2.0}    # q3 ươdng ở (1.5, 2)
    ]

# ========================================
# ẬNHP ỮD ỆLIU ỪT ƯỜNGI D NG
# ========================================
def nhap_du_lieu():
    danh_sach = []
    print("=== ƯƠCHNG TR NH T NH ỰLC ƯƠTNG T C ĨTNH ĐỆIN ===")
    while True:
        try:
            n_nhap = float(input("ậNhp ốs ượlng điện t ch trong ệh: ").strip().replace(",", "."))
            if not n_nhap.is_integer():
                print("ốS ượlng điện t ch ảphi l  ốs nguy n, v  ụd 3.")
                continue
            n = int(n_nhap)
            break
        except ValueError:
```

```
            print("Vui l ng ậnhp ố s ượlng ằbng ố s, v  ụd 3.")

    if n < 2:
        print("ệH ốthng ầcn  t ấnht 2 đệin t ch để t nh ựlc ươtng t c!")
        return []

    def nhap_so(prompt):
        while True:
            try:
                return float(input(prompt).strip().replace(",", "."))
            except ValueError:
                print("Gi  ịtr ưcha ợhp ệl; vui l ng ậnhp ại ằbng ố s.")

    for i in range(n):
        print(f"\n--- ậNhp th ng tin cho đệin t ch q{i+1} ---")
        q = nhap_so(f"  ậNhp gi  ịtr q{i+1} (Coulomb, v  ụd 2e-6 ặhoc 0.000002): ")
        x = nhap_so(f"  ậNhp ọta độ x{i+1} (m t): ")
        y = nhap_so(f"  ậNhp ọta độ y{i+1} (m t): ")
        danh_sach.append({'q': q, 'x': x, 'y': y})

    return danh_sach

# ạChy ươchng tr nh
if __name__ == "__main__":
    he_dien_tich = nhap_du_lieu()

    if he_dien_tich:
        ket_qua_tinh = tinh_luc_he_dien_tich(he_dien_tich)

        print("\n====== C NG ỨTHC T NH ======")
        print("ốVi đệin t ch q_i ịchu ựlc do ừtng đệin t ch q_j t c ụdng (j  i):")
        print("r_ij = √[(x_i - x_j)  + (y_i - y_j) ]")
        print("Fx_{→ji} = k q_i q_j (x_i - x_j) / r_ij ")
        print("Fy_{→ji} = k q_i q_j (y_i - y_j) / r_ij ")
        print("Fx_i = ổtng c c Fx_{→ji}; Fy_i = ổtng c c Fy_{→ji}")
        print("F = √(Fx  + Fy )")
        print("ếKt ảqu ươdi đ y đ  ộcng c c th nh ầphn v  t nh độ ớln ựlc.\n")

        print("====== ẾKT ẢQU T NH ỰLC ĐỆIN T C ỤDNG ỪTNG ĐỆIN T CH ======\n")
        for dien_tich_id, luc in ket_qua_tinh.items():
            print(f"--- Đệin t ch q{dien_tich_id} ---")
            for component in luc['Fx_components']:
                component_label = f"Fx{component['source_id→']}{dien_tich_id}"
                print(
                    f"{component_label} (q{component['source_id']} t c ụdng l n q{
    dien_tich_id}): "
                    f"{component['value']:.4e} N"
                )
            fx_sum = " + ".join(
                f"Fx{c['source_id→']}{dien_tich_id}" for c in luc['Fx_components']
            ) or "0"
            print(f"Fx = {fx_sum} = {luc['Fx_val']:.4e} N")

            for component in luc['Fy_components']:
                component_label = f"Fy{component['source_id→']}{dien_tich_id}"
                print(
                    f"{component_label} (q{component['source_id']} t c ụdng l n q{
    dien_tich_id}): "
                    f"{component['value']:.4e} N"
                )
            fy_sum = " + ".join(
                f"Fy{c['source_id→']}{dien_tich_id}" for c in luc['Fy_components']
            ) or "0"
            print(f"Fy = {fy_sum} = {luc['Fy_val']:.4e} N")
            print(
                f"F = {luc['F_total']:.4e} N\n"
            )
```

```
        ve_so_do_dien_tich(he_dien_tich, ket_qua_tinh)
```

Listing 1: Mã nguồn chương trình tính lực tương tác tĩnh điện

3.5 Phương pháp phân tích dữ liệu hoặc kiểm định giả thiết

- Thay thế giá trị số vào biểu thức Symbolic (phép tính `subs`).
- Kiểm định giả thiết định luật III Newton bằng cách tính tổng tổng cộng $\sum F_x$ và $\sum F_y$ của tất cả điện tích trong hệ.

4 KẾT QUẢ

4.1 Mô tả dữ liệu

Sau khi nhập mẫu dữ liệu 3 điện tích điểm vào chương trình, mã nguồn thực hiện xuất ra các biểu thức ký hiệu tổng quát và tính giá trị lực số dư tương ứng.

Bảng 4.1. Bảng dữ liệu đầu vào và kết quả thành phần lực tác dụng lên từng điện tích:

Điện tích	**Thành phần F_x (N)**	**Thành phần F_y (N)**	**Độ lớn $F_{\text{tổng}}$ (N)**
q_1 $(+2\ \mu\text{C})$	$+0{,}6000$	$-0{,}4500$	$0{,}7500$
q_2 $(-3\ \mu\text{C})$	$-0{,}1680$	$+0{,}3456$	$0{,}3843$
q_3 $(+4\ \mu\text{C})$	$-0{,}4320$	$+0{,}1044$	$0{,}4444$

4.2 Kiểm định giả thiết / Kết quả phân tích

- Kiểm định tổng lực tác dụng lên toàn hệ:

$$\sum F_x = (+0{,}6000) + (-0{,}1680) + (-0{,}4320) = 0{,}0000\ \text{N}$$
$$\sum F_y = (-0{,}4500) + (+0{,}3456) + (+0{,}1044) = 0{,}0000\ \text{N}$$

$\Rightarrow$ Giả thiết 1 hoàn toàn đúng, thỏa mãn định luật bảo toàn động lượng và định luật III Newton trong hệ kín.

4.3 Bình luận về kết quả (so sánh, đánh giá)

- **So sánh với phương pháp tính tay truyền thống:** Phương pháp Symbolic lập trình cho kết quả chính xác 100%, thời gian tính toán chỉ tốn vài millisecond so với 15–20 phút tính tay.
- **Đánh giá tính tổng quát:** Chương trình có thể mở rộng tính toán cho $n = 10, 50$ hay 100 điện tích mà không cần thay đổi cấu trúc mã nguồn.

5 KẾT LUẬN VÀ KIẾN NGHỊ

5.1 Khái quát hóa (generalization) về những điều tìm thấy

Đề tài đã hoàn thành xuất sắc các yêu cầu của Bài 10: Xây dựng thành công thuật toán tính toán lực tương tác tĩnh điện bằng phương pháp đại số ký hiệu (Symbolic Math) trên mặt phẳng Oxy.

5.2 Ý nghĩa (implication)

Khẳng định hiệu quả vượt trội của việc kết hợp giữa lý thuyết Vật lý đại cương và các công cụ tin học hiện đại trong việc giải quyết bài toán lý thuyết phức tạp.

5.3 Những hạn chế

- Bài toán mới xét trong không gian 2D (Oxy) và môi trường chân không liên tục.
- Chưa xét đến hiệu ứng phân cực trong các môi trường điện môi không đồng nhất.

5.4 Kiến nghị hướng nghiên cứu tiếp theo

- Phát triển chương trình lên không gian 3 chiều ($Oxyz$).
- Xây dựng giao diện người dùng GUI tương tác trực quan (Tkinter / PyQt).

6 TÀI LIỆU THAM KHẢO

[1] Đại học Bách Khoa TPHCM, Sổ tay hướng dẫn thực hiện luận văn tốt nghiệp đại học, 2013.

[2] A. L. Garcia and C. Penland, MATLAB Projects for Scientists and Engineers, Prentice Hall, Upper Saddle River, NJ, 1996.

[3] Python Novice Course Materials & Documentation: https://lucydot.github.io/python_novice/

[4] Visualising Data with Python / MATLAB: https://lucydot.github.io/python_novice/14-visualising-data/index.html